API Security Risk Analysis Report

Cyber Security Internship – Task 3

Summary

This report presents a read-only API Security Risk Analysis performed on a public demo API (JSONPlaceholder). The objective of this assessment was to evaluate authentication mechanisms, access controls, data exposure practices, and security configurations.

The analysis identified multiple security weaknesses including:

- Lack of authentication enforcement
- Missing object-level authorization
- Excessive data exposure
- Absence of rate limiting controls

While the tested API is intentionally open for learning purposes, similar weaknesses in production SaaS environments could result in data breaches, unauthorized access, and service disruption.

Tools Used

Postman (API Testing & Request Inspection)

- Browser Developer Tools
- Manual Risk Analysis

API Tested

API Name: JSONPlaceholder
Base URL: https://jsonplaceholder.typicode.com

Endpoints Tested:

- /users
- /users/1

Detailed Security Findings

📂 Finding 1: Unauthenticated Endpoint Access

Endpoint:

GET /users

Observation:

The endpoint allows retrieval of all user records without requiring authentication, API keys, or authorization tokens.

No login mechanism or header-based authentication was observed.

Risk Explanation:

In a real SaaS application, this would allow attackers to scrape or collect user data without credentials.

Severity:

Medium (High if used in production with real user data)

Business Impact:

- Data scraping
- Privacy violations
- Regulatory compliance risks (GDPR, etc.)
- Increased attack surface

Recommendation:

- Implement JWT or OAuth 2.0 authentication
- Enforce authentication middleware on all sensitive endpoints
- Restrict anonymous API access

📂 Finding 2: Excessive Data Exposure

Endpoint:

GET /users

Observation:

The API response includes:

- Full name

- Email address
- Phone number
- Address
- Geo-location
- Company details

Many of these fields may not be required for normal application functionality.

Risk Explanation:

Returning unnecessary sensitive fields increases risk of data harvesting and misuse.

Severity:

Medium

Business Impact:

- Sensitive information leakage
- Increased phishing risk
- Data aggregation by attackers

Recommendation:

- Implement response filtering
- Apply "least data exposure" principle
- Return only fields required for specific functionality

📂 Finding 3: Missing Object-Level Authorization (BOLA Risk)

Endpoint:

GET /users/{id}

Example:

- /users/1
- /users/2

Observation:

The API allows direct access to specific user records using only an ID parameter.

There is no verification of requester identity.

Risk Explanation:

In real-world systems, this vulnerability is known as Broken Object Level Authorization (BOLA), which is one of the OWASP API Top 10 risks.

Attackers could change the ID value to access other users' data.

Severity:

Medium (High in production SaaS systems)

Business Impact:

- Unauthorized access to user data
- Data breach incidents
- Legal consequences
- Loss of customer trust

Recommendation:

- Implement user identity verification
- Enforce access control checks
- Ensure users can only access their own records

📂 Finding 4: Missing Rate Limiting & Throttling

Observation:

No rate limiting headers were observed in responses.

There is no visible restriction on number of requests per client.

Risk Explanation:

Without rate limiting, attackers could:

- Send unlimited automated requests
- Attempt brute-force enumeration
- Perform scraping attacks
- Overload backend systems

Severity:

Medium

Business Impact:

- Service disruption
- Increased infrastructure cost
- Potential denial-of-service scenarios

Recommendation:

- Implement API throttling

- Use request rate limits per IP/user
- Configure API gateway protections

Risk Severity Summary

Risk	Severity
Unauthenticated Access	Medium
Excessive Data Exposure	Medium
Missing Object Authorization	Medium (High in SaaS)
Missing Rate Limiting	Medium

Overall Security Assessment

The tested API lacks essential security controls such as authentication enforcement, object-level authorization, and rate limiting.

Although this API is designed for testing and educational purposes, similar misconfigurations in real-world SaaS applications could lead to significant security incidents.

Modern applications should implement layered API security including:

- Authentication
- Authorization
- Input validation
- Data minimization
- Monitoring and logging
- Rate limiting

Conclusion

This assessment demonstrates practical understanding of API security risks in modern SaaS environments. The identified vulnerabilities align with common real-world API weaknesses highlighted in industry security frameworks.

Proper implementation of authentication, access control, and request monitoring is critical to securing APIs in production systems.